CSE493L 26Wi Practice Exam

This practice exam consists of 31 multiple choice questions, followed by four FRQs, which should take approximately 110 minutes to complete.

Every question is worth 1 point, except for the free response questions which are worth 3 points each.

Key:

◯ = exactly one answer should be selected

☐ = multiple answers may be selected

Please fill in the square or circle fully.

Multiple Choice Questions

1) Consider the following code snippet, which creates multiple coroutines. You can assume these coroutines function in the same way as we talked about in class, meaning that a call to `send`, `recv`, `yield`, or `spawn` could switch execution to any coroutine in the ready queue.

```
void main() {
    h2, c2 = spawn(CR2)
    send(c2, null)
    print("1")
    join(h2)
}

void CR2(c2) {
    h3, c3 = spawn(CR3)
    send(c3, c2)
    recv(c2)
    print("2")
    join(h3)
}

void CR3(c3) {
    c2 = recv(c3)
    recv(c2)
    print("3")
}
```

Choose all possible outputs of this program:

- ☐ 13
- ☐ 123
- ☐ 132
- ☐ 21
- ☐ 32

2) Why do HW1 coroutines perform the context switch in assembly as opposed to in C? Select all that apply.

- ☐ Changing the stack pointer is impossible in C.
- ☐ Backing up and restoring individual registers is impossible in C.
- ☐ Putting the instruction pointer on the stack is impossible in C.

3) Logically, what is necessary to save, at a minimum, to backup and restore state in a HW1 coroutine switch? Select all that apply.

- ☐ Cache coherence state
- ☐ Registers
- ☐ Text segment
- ☐ The program counter
- ☐ Stack
- ☐ Heap

4) What does adding the volatile keyword to a variable declaration mean?

- ◯ Reads and writes to this variable will not be reordered with respect to other reads and writes to this variable.
- ◯ All reads and writes to this variable must be issued and not optimized away.
- ◯ This variable will be stored on its own cache line, making it quicker to update
- ◯ Read and writes to this variable will be performed without using the cache

5) Which of the following statements about concurrency and parallelism are true? Select all that apply.

- ☐ Concurrency means multiple things could be done, parallelism means multiple things are being done.
- ☐ Concurrency is a property of the runtime, parallelism is a property of the dependency graph.
- ☐ Concurrency is a programming paradigm, parallelism is a performance optimization.
- ☐ Parallelism is impossible unless the problem is partitionable into multiple concurrent operations.

6) What is Gustafson's law trying to argue? When is it more appropriate than Amdahl's law?

- ◯ Typically the amount of work scales with the amount of resources; when the work size is fixed.
- ◯ Typically the amount of work scales with the amount of resources; when the work is easily decomposable.
- ◯ Throwing more cores at the same problem yields a linear speedup; when the work is embarrassingly parallel.
- ◯ Throwing more cores at the same problem yields a linear speedup; when the work size can grow.

7) We do not need barriers if we are guaranteed that cores run in lockstep.

- ◯ True, all cores will run at the same speed and have implicit barriers between instructions.
- ◯ False, instructions can take different numbers of clock cycles to execute
- ◯ False, we will need them to synchronize if we have more threads than cores.
- ◯ False, we need them to prevent the CPU from reordering memory operations

8) Why are there no data races in Rust? Select all that apply.

- ☐ There are no null references.
- ☐ The ownership system prevents use after frees.
- ☐ Shared mutable state is only accessible through using synchronization primitives, eliminating data races.
- ☐ The borrowing rules prevent two mutable borrows.

9) What is a critical section?

- ◯ A section of code that a thread cannot be preempted within while executing
- ◯ A section of code that modifies at least one object through a mutable reference.
- ◯ A lock.
- ◯ A section of code that only one thread can be running at a time.

10) If your code always runs on 1 core, spinlocks are not a helpful tool to synchronize threads.

- ◯ True
- ◯ False

11) Data races are the only source of race conditions.

- ◯ True
- ◯ False

12) What is the simplest implementation of a critical section with…

a) … cooperative (non-preemptable), multicore routines?
- ◯ Disable interrupts
- ◯ Use a SpinLocks to protect the critical section
- ◯ Use an Interrupt Safe SpinLock to protect the critical section

b) … preemptive, multicore routines?
- ◯ Disable interrupts
- ◯ Use a SpinLocks to protect the critical section
- ◯ Use an Interrupt Safe SpinLock to protect the critical section

c) … preemptive, single core routines?
- ◯ Disable interrupts
- ◯ Use a SpinLocks to protect the critical section
- ◯ Use an Interrupt Safe SpinLock to protect the critical section

13) Consider this following code snippet, that runs on a thread:

```
mutex.lock();
while (x == 0) {
    mutex.unlock();
    condvar.wait(nullptr);
    mutex.lock();
}
assert(x != 0);
```

At some point, another thread sets `x` to 5. Is it possible for our thread to sleep forever?

- ◯ No, because we recheck the condition on each iteration of the while loop.
- ◯ No, because the other thread will wake us by notifying the condition variable.
- ◯ Yes, because the other thread might set x to 5 after we unlock, but before we wait.
- ◯ Yes, because condition variables have spurious wake ups that we didn't account for here.

14) How does Rust know when to call destructors?

- ◯ When the owner goes out of scope.
- ◯ When the last reference goes out of scope.
- ◯ When any reference goes out of scope.
- ◯ When ownership is transferred.

15) If R is the number of shared references to `x`, and W is the number of mutable borrows to `x` at some program point. Select all assignments to R and W that the Rust compiler would allow.

- ☐ R=0, W=0
- ☐ R=1, W=1
- ☐ R=2, W=0
- ☐ R=2, W=2
- ☐ R=1, W=2
- ☐ R=2, W=1
- ☐ R=1, W=0
- ☐ R=0, W=1
- ☐ R=0, W=2

16) Consider the following function signature:

```
fn longest<'a>(s1: &'a str, s2: &'a str) -> &'a str {...}
```

What statements are true, regarding the lifetime `'a`? Select all that apply.

- ☐ The function's returned value cannot outlive either s1 or s2.
- ☐ The function's returned value must outlive both s1 and s2.
- ☐ The function can always just return a static (global) string.
- ☐ The caller must ensure that the returned string is dropped before either of the inputs to longest.
- ☐ The caller cannot make a mutable reference to s1 or s2 while the returned value from longest is still in scope.

17) Trait bounds allow us to specify that some functions can only be called with types that implement certain traits. What happens when we call a function twice with two different types that both satisfy the bound?

- ◯ The compiler generates code to figure out which copy of the function to run at runtime.
- ◯ The compiler generates code for one of the function calls, then casts the other type into the first before calling the function.
- ◯ The compiler generates two copies of the function, one for each type.

18) MutexGuard is Sync but not Send. Which of the following statements are true? Select all that apply.

- ☐ We cannot put a MutexGuard into an Arc.
- ☐ We can share references to a MutexGuard between threads, but we cannot transfer ownership of it between threads.
- ☐ We can transfer ownership of a MutexGuard between threads, because it only contains a reference to locked data.
- ☐ Sync implies not Send.

19) Consider the following code snippet:

```rust
struct Vec2 {
    x: f64,
    y: f64,
}
impl Vec2 {
    fn scale(self: Vec2, factor: f64) -> Vec2 {
        Vec2 {
            x: self.x * factor,
            y: self.y * factor,
        }
    }
}
fn main() {
    let v = Vec2 { x: -1.0, y: 5.0 };
    v.scale(2.0);
    println!("The upscaled Vec2 is: {} {}", v.x, v.y);
}
```

Select all that apply.

- ☐ This doesn't compile because we're not allowed to access both `x` and `y` in the same scope.
- ☐ This doesn't compile because ownership of `v` is moved into `scale()`, and then `v` cannot be reused.
- ☐ We could fix this by adding a match statement to exhaustively handle all possible values of `v`.
- ☐ We could fix this by making `v` mutable and rewriting `scale()` to update `v` by accepting a mutable reference to it.
- ☐ We could fix this by assigning the result of `v.scale()` to another variable, and printing that.

20) Select all necessary conditions for deadlock.

- ☐ Limited resources
- ☐ No preemption
- ☐ Hold and wait
- ☐ Cyclical dependencies

21) Assume MSI. Core 1's cache has cache line 5 in the M (modified) state. It overhears core 0's cache is trying to fetch cache line 5 in the M (modified) state. What state should core 0's cache transition to, and should a writeback to memory occur?

- ◯ I; Writeback should not occur
- ◯ S; Writeback should not occur
- ◯ I; Writeback should occur
- ◯ S; Writeback should occur

22) Assume the following cores execute the given instructions in parallel:

```
// CORE 0:          // CORE 1:
x = 1               y = 1
%r0 = y             %r1 = x
```

a) What are the final possible MSI states of x and y? Assume x and y begin in the invalid state, and exist on separate cachelines, in a SeqCst memory consistency model. Select all that apply.

- ☐ x: Shared, y: Shared
- ☐ x: Modified, y: Shared
- ☐ x: Shared, y: Modified
- ☐ x: Modified, y: Modified

b) Now, assume Total Store Ordering consistency (W->R no longer enforced, reordering possible). What states are possible now?

- ☐ x: Shared, y: Shared
- ☐ x: Modified, y: Shared
- ☐ x: Shared, y: Modified
- ☐ x: Modified, y: Modified

23) Select all correct statements about thread pools and event-based programming.

- ☐ Event-based programming can be implemented on a thread pool.
- ☐ Thread pools are faster than threads at running code in parallel.
- ☐ Event-based programming has more interfaces with the userspace scheduler than thread pools do.
- ☐ Async/await is just language-level support (syntactic sugar) for event-based programming.

24) If we have `let x = &mut a` and it compiles, what invariants do we know about x? Select all that apply.

- ☐ `x` is aligned, non-null, and valid for lifetime `'static`
- ☐ `x` is the only reference to `a`
- ☐ `x` references data that is safe for `a`'s data type
- ☐ `x` references data in either the stack or the heap

25) How do GPUs offer thousands of cores, while CPUs can only offer tens? Select all that apply.

- ☐ GPUs have a severely restricted programming model.
- ☐ GPU cores are slower than CPU cores.
- ☐ GPUs are optimized for floating point math, which is faster than CPU's integer maths.
- ☐ GPU cores in a warp can only run the same instructions in lockstep.

26) Select the incorrect statement about batch processing systems and stream processing systems.

- ○ Batch processing systems reuse resources between operations, and stream processing systems dynamically find a free resource to use.
- ○ In order to balance utilization, both systems pre-shard the input dataset to distribute it to the workers more easily.
- ○ Upon failure, stream processing systems are simple to recover from, since lineage is implicit in their partitioning strategy, whereas batch systems have to log or checkpoint.

27) Parallelism in ML models has many different flavors with various tradeoffs. What do pipeline parallelism and tensor parallelism always have in common?

- ◯ Both decrease the overall latency of computation.
- ◯ Both distribute the weights over multiple GPUs.
- ◯ Both scale linearly.
- ◯ Both make use of point-to-point communication rather than collective communication

28) Achieving peak performance on GPUs often involves utilizing shared memory. Why is this difficult compared to CPU memory usage?

- ☐ Shared memory is managed by the program, but CPU caches are managed by the hardware.
- ☐ Shared memory is slower than the GPU caches, so it is difficult to make it performant.
- ☐ We must avoid false sharing of shared memory.
- ☐ Shared memory is small, and so it is important to use it only for data you know will be frequently accessed and shared.

29) Assume `x` is a variable in memory that starts initialized to zero. Consider this pseudocode assembly snippet. Assume total store ordering. Assume this program is run simultaneously on two cores.

```
do 100 times {
    reg ← x
    reg ← reg + 1
    x   ← reg
}
```

a) What is the smallest possible final value of the variable x?

- ◯ 1
- ◯ 2
- ◯ 3
- ◯ 99
- ◯ 100

b) What is the largest possible final value of the variable x?

- ◯ 100
- ◯ 199
- ◯ 200
- ◯ 201

c) What assembly instructions are required to make the final value deterministic?

- ◯ Fences
- ◯ Interrupt masking
- ◯ Atomics

30) Why do we need `std::mem::swap`? Select all that apply.

- ☐ To save memory versus a traditional, 3 variable swap.
- ☐ To swap the values of two mutable references when we don't have ownership/permission to move out of the value.
- ☐ To implement Option::take

31) Assume a multicore CPU where each core has its own cache. The state of each cache line in a cache is either M (modified), S (shared), or I (invalid) state.

Draw a checkmark (✓) in allowed combinations, and a cross (✗) in disallowed combinations:

		State of a cache line in cache 0:		
		M	**S**	**I**
State of the same cache line in cache 1:	**M**			X
	S		X	X
	I	X	X	X

Free Response Questions

1) Come up with a race condition that is not dependent on a data race. Write it out in pseudocode. Optional bonus point: come up with a data race that is not a race condition.

Answers:
Race condition that is not a data race:

```
        Thread 1:
                *mutex.lock().unwrap() = 1;
        Main thread:
                mutex = Mutex::new(0);
                spawn(thread 1);
                println!("value is {}", mutex.lock().unwrap());
```

Data race that is not a race condition (in assembly pseudocode):

```
        Thread 1:
                x++;
        Thread 2:
                x++;
        Main thread:
                global x = 0
                spawn(thread 1);
                spawn(thread 2);
                // ignore thread1, thread2, and x for the rest of the program
```

2) Give an example of a time you'd use a Rust enum over a trait, and vice versa. Justify your choices.

Answers:
Enums: If the interface is defined by the types that implement it, if you know all the types variants, when you need pattern matching. Example: a point *is* either cartesian or polar, so it is an enum.
Traits: if you want external code to be able to add variant types, no access to inner data, defines shared behavior Example: an iterator *can do* these things, so it is a trait.

3) Say you are implementing a distributed prefix sum that is running on a fleet of GPUs. Each GPU can run a parallel prefix sum locally.

a. You have many, many tiny arrays you want to sum quickly. How might you parallelize the work across all the machines? Why?
b. You have one giant array you want to sum quickly, but it's too big to fit on one machine. How might you parallelize the work across all the machines? Why?
c. Can you combine the above two approaches? When is this useful?

Answers:

a. You can distribute the arrays, 1 per GPU, to achieve more throughput but equal latency per item.
b. You can distribute the giant array, and then perform collective communication to reduce across GPUs (similar to tensor parallelism). This may take several phases, but could also be pipelined (similar to pipeline parallelism). This will give lower overall latency assuming sufficient network bandwidth.
c. Yes, you can combine the two approaches by distributing many large arrays to clusters of GPUs that each run in tensor and pipeline parallel. This is always useful to achieve peak performance, especially when you have a heterogeneous network (e.g. some nodes close, some nodes far).

4) In Rust, you can write the following code:

```
enum MyWeirdEnum {}
struct MyStrangeStruct {}
```

Note, the above enum does not have any variants. The struct does not have any fields. Pick one of these two types and describe why you'd ever want to use it.

Answers:

MyWeirdEnum has no variants, so it cannot be constructed. However, it can still be used as a generic parameter and can still implement traits to implement e.g. compile-time enforced ordered locking. It could also be used e.g. in a Result error variant (Result<i32, MyWeirdEnum>) to specify that the Err can never happen, since it cannot be constructed.

MyStrangeStruct has no elements, and so it can be constructed, but is equivalent to the unit type. It can therefore be passed into functions as a "token" with zero runtime cost, and could be used to perform resource accounting at compile time.